

M5Stack / Arduino Exam Cheat Sheet

Everything in this PDF matches the example .ino files. The exam board is an **M5Stack** (ESP32, 320x240 screen, buttons A/B/C, speaker, LED on pin 21 in these questions). Serial Monitor baud rate: **115200**.

1. Minimum program template

```
#include <M5Stack.h>          // always first line

int x = 5;                    // GLOBAL variable (shared by setup and loop)

void setup() {                // runs ONCE
    M5.begin();               // starts screen, speaker, buttons, Serial(115200)
    Serial.begin(115200);     // optional, makes it explicit
    pinMode(21, OUTPUT);      // only if you use a pin
}

void loop() {                 // runs FOREVER
    M5.update();              // MUST be here, refreshes button states
    // your code
}
```

2. M5Stack function reference

Buttons (BtnA, BtnB, BtnC)	Meaning
M5.BtnA.wasPressed()	TRUE once, at the moment of pressing (use this for counters)
M5.BtnA.wasReleased()	TRUE once, when let go
M5.BtnA.isPressed()	TRUE the whole time it is held (counter would race!)
M5.BtnA.pressedFor(1000)	TRUE if held at least 1000 ms
Screen (M5.Lcd)	Meaning
setTextSize(n)	Text size, 1 = small, 10 = huge (n=10 fits about 5 digits across)
setTextColor(GREEN) / (0x07E0)	Text color. Also RED BLUE WHITE BLACK YELLOW CYAN ORANGE. 0x07E0 = green
setTextColor(fg, bg)	Text color + background color
setCursor(x, y)	Where the next text appears. (0,0) = top-left. Screen is 320 x 240
print(v) / println(v)	Draw text/number at cursor. println moves to next line
clear() / fillScreen(BLACK)	Wipe the screen. ALWAYS setCursor again after clearing
drawString("txt", x, y)	Text at exact position
drawRect / fillRect(x,y,w,h,col)	Rectangle outline / filled
drawCircle / fillCircle(x,y,r,col)	Circle outline / filled
drawLine(x1,y1,x2,y2,col)	Line
color565(r,g,b)	Make custom color from 0-255 RGB
Speaker (M5.Speaker)	Meaning
tone(freq)	Start a tone and keep playing until mute() is called
tone(freq, ms)	Play a tone for ms milliseconds
mute()	Stop sound. Standard pattern: tone(f); delay(500); mute();
setVolume(n)	Volume (small numbers like 1-8 are fine)
Note frequencies (Hz)	C4=262 D=294 E=330 F=349 G=392 A=440 B=494 C5=524 low c=131
Pins and Serial	Meaning
pinMode(21, OUTPUT)	Set pin as output (do once in setup)
digitalWrite(21, HIGH/LOW)	LED on / off
Serial.available()	How many characters are waiting. >0 means something was typed
Serial.read()	Read ONE character (returns int, compare with 'o' in single quotes)

Pins and Serial	Meaning
<code>Serial.write(key)</code>	Send one character back (echo)
<code>Serial.println("text")</code>	Print a line to Serial Monitor
<code>Serial.readStringUntil('\n')</code>	Read a whole line as a String (then <code>.trim()</code> it)
<code>Serial.parseInt()</code>	Read a number typed in
<code>delay(ms)</code>	Pause everything for ms milliseconds (blocks buttons!)
<code>millis()</code>	Milliseconds since start. Use for non-blocking timing

3. C/Arduino essentials

```

if (x > 0) { ... } else if (x == 0) { ... } else { ... }

while (x < 10) { x++; }           // repeats while condition true
for (int i = 0; i < 3; i++) { ... } // runs 3 times (i = 0,1,2)

switch (key) {                   // compare key with single chars
  case 'o':                      // stacked cases share code:
  case '0':
    digitalWrite(21, HIGH);
    break;                      // DON'T forget break
  default:                      // everything else
    digitalWrite(21, LOW);
    break;
}

void myFunction(int value) { ... } // define OUTSIDE setup/loop, call by name
ledState = !ledState;             // ! flips HIGH<->LOW (true<->false)
x++; x--; x += 2;                 // change a number
#define C 524                     // constant name (no = and no ;)
'o' = a character (single quotes) "on" = a String/text (double quotes)

```

Operator	Meaning	Operator	Meaning
=	ASSIGN (put value in)	==	COMPARE (are they equal?)
!=	not equal	<= >=	less/greater or equal
&&	AND		OR

4. Bug checklist (the mistakes in the original answers)

Mistake	Why it is wrong	Fix
if (x = 10)	= assigns 10 to x, condition is always TRUE	if (x == 10)
if (x = 0)	sets x to 0 EVERY time and is always FALSE, so the block never runs	if (x == 0)
int x = 5; inside a block	creates a NEW local x that hides the global; global stays unchanged	x = 5; (no int)
Button C { } placed inside the Button B if { }	C is only checked when B is pressed, so effectively never	Put each button's if at the same level in loop()
Forgot M5.update()	Buttons never update	First line of loop()
Forgot setCursor after clear()	Text appears at a stale/old position	clear(); setCursor(0,0);
Forgot break in switch	Falls through into the next case	break; at end of every case
'o' vs "o"	Single quote = char, double quote = text	Use 'o' with Serial.read()
Serial Monitor sends \n after Enter	\n lands in default and turns LED off	case '\n': case '\r': break;
ledState out of sync with LED	Written to the pin but not to the variable, toggle breaks	Use helper setLed()

5. EXAM Q1: counter with limits (explained)

Task: screen shows 5 (big, green). **A:** count up to 10 (one number per 500 ms), then high tone. **B:** count down; at 0 play low tone. **C:** reset to 5 with middle tone.

Idea: one global variable x is the counter. Every button changes x, redraws it (clear, setCursor, print) and may beep. The while loop in A blocks until x reaches 10, so numbers appear one by one. Tones: C=524, c=131, C1=262 Hz.

```
#include <M5Stack.h>

#define C  524  // high tone  (reached 10)
#define c  131  // low tone   (reached 0)
#define C1 262  // middle tone (reset)

int x = 5;      // global counter

// Clear the screen and draw the current value of x
void showNumber() {
    M5.Lcd.clear();
    M5.Lcd.setCursor(0, 0);
    M5.Lcd.println(x);
}

// Play a tone for half a second, then silence
void beep(int freq) {
    M5.Speaker.tone(freq);
    delay(500);
    M5.Speaker.mute();
}

void setup() {
    M5.begin();
    M5.Lcd.setTextSize(10);
    M5.Lcd.setTextColor(0x07E0); // green
    showNumber();                // shows 5
}

void loop() {
    M5.update();                // refresh button states

    // ----- Button A: count up to 10 -----
    if (M5.BtnA.wasPressed()) {
        while (x < 10) {
            x++;
            showNumber();
            delay(500);
        }
        if (x == 10) {          // == compares, = would assign
            beep(C);
        }
    }

    // ----- Button B: count down to 0 -----
    if (M5.BtnB.wasPressed()) {
        if (x > 0) {
            x--;
            showNumber();
        }
        if (x == 0) {
            beep(c);
        }
    }

    // ----- Button C: reset to 5 -----
    if (M5.BtnC.wasPressed()) {
        x = 5;                  // no "int": we change the GLOBAL x
        showNumber();
        beep(C1);
    }
}
```

6. EXAM Q2: LED controlled from Serial Monitor (explained)

Task: read a typed character. **o/O** LED on, **t/T** toggle, **b/B** blink 3 times then stay on, any other key LED off.

Idea: check `Serial.available()`, read one char with `Serial.read()`, then use `switch`. Stack upper/lower case labels. Keep `ledState` updated every time the LED changes so toggle works. Blink = for loop of (HIGH, delay 500, LOW, delay 500) then HIGH.

```
#include <M5Stack.h>

int ledPin = 21;
int ledState = LOW;

// Write a state to the LED AND remember it in ledState
void setLed(int state) {
    ledState = state;
    digitalWrite(ledPin, ledState);
}

// Blink 3 times (500 ms on / 500 ms off), then leave the LED ON
void blinkThree() {
    for (int i = 0; i < 3; i++) {
        setLed(HIGH);
        delay(500);
        setLed(LOW);
        delay(500);
    }
    setLed(HIGH);
}

void setup() {
    M5.begin();
    Serial.begin(115200);
    pinMode(ledPin, OUTPUT);
    setLed(LOW);
}

void loop() {
    M5.update();

    if (Serial.available() > 0) {
        int key = Serial.read();
        Serial.write(key);           // echo the key back

        switch (key) {
            case 'o':
            case 'O':
                setLed(HIGH);
                break;

            case 'b':
            case 'B':
                blinkThree();
                break;

            case 't':
            case 'T':
                setLed(!ledState);    // flip the LED
                break;

            case '\n':
            case '\r':                // ignore Enter / newline characters
                break;

            default:
                setLed(LOW);          // any other key: LED off
                break;
        }
    }
}
```

Alternative blink without a for loop (what the second version did): LOW, wait, HIGH, wait, LOW, wait, HIGH, wait, LOW, wait, HIGH, wait, LOW, wait, HIGH (7 delays then stays on).

7. Likely variations: quick snippets

```
// Show a different start value / limit
#define MAX_VAL 20          // change 10 to 20; change while (x < 20)

// Count up by 2
x += 2;

// Wrap around instead of stopping (9 -> 0)
x++; if (x > 9) x = 0;

// Count up/down while button is HELD (use isPressed + small delay)
if (M5.BtnA.isPressed()) { x++; showNumber(); delay(200); }

// Hold button C for 1 second to reset
if (M5.BtnC.pressedFor(1000)) { x = 5; showNumber(); }

// Change text color depending on value
if (x >= 8) M5.Lcd.setTextColor(RED); else M5.Lcd.setTextColor(GREEN);

// Number + text on screen
M5.Lcd.print("Count: "); M5.Lcd.println(x);

// Print to Serial Monitor too
Serial.print("x = "); Serial.println(x);

// Beep with different tone per button
M5.Speaker.tone(262); delay(200); M5.Speaker.mute();

// LED: on for 2 seconds then off
digitalWrite(21, HIGH); delay(2000); digitalWrite(21, LOW);

// LED blink N times (function)
void blinkN(int n, int ms) {
  for (int i = 0; i < n; i++) {
    digitalWrite(21, HIGH); delay(ms);
    digitalWrite(21, LOW);  delay(ms);
  }
}

// Serial: add more commands to the switch
case 'f': case 'F': digitalWrite(ledPin, LOW); ledState = LOW; break;
case 's': case 'S': Serial.println(ledState ? "ON" : "OFF"); break;
```

8. Guide to the example .ino files

File	Topic
EXAM_Q_1_fixed.ino	Q1 counter: A up, B down, C reset, tones (the corrected exam answer)
EXAM_Q_2_fixed.ino	Q2 Serial LED: o/t/b commands (the corrected exam answer)
01_buttons_basic	wasPressed / isPressed / wasReleased, colors, clear
02_counter_limits_tones	General counter with MIN/MAX constants and tones
03_serial_led_commands	Single-character serial commands incl. f for off
04_serial_words_and_numbers	Whole-word commands, readStringUntil, toInt, substring
05_button_controls_led	Buttons control LED and print state to screen + Serial
06_blink_without_delay_millis	Non-blocking timing with millis()
07_tones_and_melody	Note arrays, for loop melody, volume
08_screen_drawing	Shapes, text sizes, colors, drawString

Tip: in Arduino IDE, opening a lone .ino asks to move it into a folder of the same name. Click OK. Select board **M5Stack-Core-ESP32** (or what the lab uses) and the correct port before Upload.